```rust
fn main() {
  let numbers = [1, 2, 3, 4, 5];
  println!("The first number is: {}",
numbers[0]);
}
```

```
The first number is: 1
```

```rust
fn main() {
  let mut numbers = [1, 2, 3, 4, 5];
  numbers[0] = 10;
  println!("The new first number is:
{}", numbers[0]);
}
```

```
The new first number is: 10
```

```rust
fn main() {
  let fruits = ["apple", "banana",
"orange"];
  for fruit in fruits {
    println!("I like {}.", fruit);
  }
}
```

```
I like apple.
I like banana.
I like orange.
```

```rust
fn main() {
  let numbers = [1, 2, 3, 4, 5];
  println!("{:?}", numbers);
}
```

```
[1, 2, 3, 4, 5]
```

Code

```
First fruit: apple
```

Code

```
New first fruit: grape
```

Code

```
["apple", "banana", "cherry"]
```

Code

```
["apple", "banana"]
```

Code

```
["apple", "banana", "orange"]
```

Code

```
["banana", "apple", "orange"]
```

Code

```
["banana", "orange"]
```

Code

```
There are 3 fruits.
```

Code

```
I like apple.
I like banana.
I like orange.
```

```rust
fn get_user() -> (String, i32) {
  (String::from("Liam"), 25)
}

fn main() {
  let user = get_user();
  println!("User: {} ({} years old)",
user.0, user.1);
}
```

```
User: Liam (25 years old)
```

```rust
fn main() {
  let person = ("John", 30, true);
  println!("Name: {}", person.0);
  println!("Age: {}", person.1);
  println!("Is active: {}", person.2);
}
```

```
Name: John
Age: 30
Is active: true
```

```rust
use std::collections::HashMap;

fn main() {
  // Create a HashMap called capitalCities
  let mut capitalCities = HashMap::new();

  // Add keys and values (Country, City)
  capitalCities.insert("England", "London");
  capitalCities.insert("Germany", "Berlin");
  capitalCities.insert("Norway", "Oslo");

  println!("{:?}", capitalCities);
}
```

```
{"England": "London", "Germany": "Berlin", "Norway": "Oslo"}
```

```rust
use std::collections::HashMap;

fn main() {
  let mut capitalCities = HashMap::new();

  capitalCities.insert("England", "London");
  capitalCities.insert("Germany", "Berlin");
  capitalCities.insert("Norway", "Oslo");

  if let Some(city) = capitalCities.get("England") {
    println!("The capital of England is {}.", city);
  } else {
    println!("England is not in the map.");
  }
}
```

```
The capital of England is London.
```

```rust
use std::collections::HashMap;

fn main() {
  let mut capitalCities =
HashMap::new();

  capitalCities.insert("England",
"London");
  capitalCities.insert("England",
"Berlin");

  println!("{:?}", capitalCities);
}
```

```
{"England": "Berlin"}
```

```rust
use std::collections::HashMap;

fn main() {
  let mut capitalCities =
HashMap::new();

  // Add keys and values (Country,
City)
  capitalCities.insert("England",
"London");
  capitalCities.insert("Germany",
"Berlin");
  capitalCities.insert("Norway",
"Oslo");

  // Remove the key "England"
  capitalCities.remove("England");

  println!("{:?}", capitalCities);
}
```

```
{"Germany": "Berlin", "Norway": "Oslo"}
```

```rust
use std::collections::HashMap;

fn main() {
  let mut capitalCities =
HashMap::new();

  // Add keys and values (Country,
City)
  capitalCities.insert("England",
"London");
  capitalCities.insert("Germany",
"Berlin");
  capitalCities.insert("Norway",
"Oslo");

  // Loop through the HashMap
  for (country, city) in &capitalCities
{
    println!("The capital of {} is
{}.", country, city);
  }
}
```

```
The capital of England is London.
The capital of Germany is Berlin.
The capital of Norway is Oslo.
```

```rust
fn main() {
  // Create a Struct called Person
  struct Person {
    name: String,
    age: u32,
    can_vote: bool,
  }

  // Create a Person object
  let user = Person {
    name: String::from("John"),
    age: 35,
    can_vote: true,
  };

  // Access and print the values
  println!("Name: {}", user.name);
  println!("Age: {}", user.age);
  println!("Can vote? {}",
user.can_vote);
}
```

```
Name: John
Age: 35
Can vote? true
```

```rust
fn main() {
  struct Person {
    name: String,
    age: u32,
  }

  let mut user = Person {
    name: String::from("John"),
    age: 35,
  };

  user.age = 36;  // Change value of age
  println!("Name: {}", user.name);
  println!("Updated age: {}", user.age);
}
```

```
Name: John
Updated age: 36
```

```rust
enum Direction {
  Up,
  Down,
  Left,
  Right,
}

fn main() {
  let my_direction = Direction::Left;

  match my_direction {
    Direction::Up => println!("Going up"),
    Direction::Down => println!("Going down"),
    Direction::Left => println!("Going left"),
    Direction::Right => println!("Going right"),
  }
}
```

```
Going left
```

```rust
enum LoginStatus {
  Success(String),
  Error(String),
}

fn main() {
  let result1 =
LoginStatus::Success(String::from("Welc
ome, John!"));
  let result2 =
LoginStatus::Error(String::from("Incorr
ect password"));

  match result1 {
    LoginStatus::Success(message) =>
println!("Success: {}", message),
    LoginStatus::Error(message) =>
println!("Error: {}", message),
  }
}
```

```
Success: Welcome, John!
```